

Projeto Completo — Jogo de Cards Estilo Super Trunfo com NFTs

Visão Geral

O projeto consiste em uma plataforma multiplataforma (Web, Android e iOS) para um jogo competitivo de cards inspirado no Super Trunfo.

Os cards representam personagens únicos em formato NFT, contendo atributos que influenciam diretamente nas partidas.

O sistema inclui:

- Sistema de partidas PvP e PvE
- Matchmaking inteligente
- Loja de cards
- Geração procedural de cards
- Sistema de ranking
- Economia baseada em créditos
- Expiração de cards
- Recursos sociais
- Arquitetura DDD (Domain-Driven Design)
- Estratégia de desenvolvimento ágil
- Testes automatizados
- Deploy escalável em nuvem

1. Stack Tecnológica Recomendada

Frontend

Mobile

- Flutter
- Android
- iOS

Web

- React + Next.js

Gerenciamento de Estado

- Redux Toolkit
- React Query

UI/UX

- Material UI
 - TailwindCSS
 - Framer Motion
-

Backend

Linguagem

- Python

Framework

- FastAPI

Justificativa

- Excelente performance
 - Fácil modularização
 - Ótimo suporte para APIs assíncronas
 - Integração simples com IA e geração procedural
 - Excelente produtividade
-

Banco de Dados

Relacional

- PostgreSQL

Cache / Matchmaking / Filas

- Redis

Busca

- OpenSearch
-

Mensageria

- RabbitMQ

ou

- Apache Kafka
-

Infraestrutura

- Docker
 - Kubernetes
 - Terraform
 - GitHub Actions
 - ArgoCD
-

Cloud

Recomendado

- AWS

Serviços

- EKS
 - RDS PostgreSQL
 - ElastiCache Redis
 - S3
 - CloudFront
 - Cognito
 - Lambda
 - EventBridge
-

2. Arquitetura Geral

Arquitetura DDD + Hexagonal

Contextos de Domínio

1. Identity Context

Responsável por:

- Usuários
- Login
- Friends
- Perfil
- Social

2. Card Context

Responsável por:

- Cards

- NFTs
- Geração procedural
- Expiração
- Raridade
- Coleção

3. Match Context

Responsável por:

- Matchmaking
- Sessão de jogo
- Regras
- Rodadas
- Resultado

4. Economy Context

Responsável por:

- Créditos
- Loja
- Compras
- Inventário

5. Ranking Context

Responsável por:

- Ranking global
- ELO/MMR
- Temporadas

6. Notification Context

Responsável por:

- Push notifications
- Convites
- Alertas

3. Estrutura de Pastas (DDD)

```
src/
├── modules/
│   ├── identity/
│   ├── cards/
│   ├── matchmaking/
│   └── gameplay/
```

```
|   |   | economy/
|   |   | ranking/
|   |   | social/
|   |
|   |--- shared/
|   |   | domain/
|   |   | infrastructure/
|   |   | application/
|   |   | utils/
|   |
|   |--- tests/
|   |   | unit/
|   |   | integration/
|   |   | e2e/
|   |
|--- main.py
```

4. Modelagem dos Cards

Regras

- Nunca podem existir dois cards idênticos
- Cada card possui:
 - UUID único
 - Hash de atributos
 - NFT Token ID
- Cards podem pertencer à mesma família
- Similaridade parcial é permitida
- Igualdade total é proibida

5. Estrutura de Dados do Card

```
from dataclasses import dataclass
from uuid import UUID
from datetime import datetime

@dataclass
class Card:
    id: UUID
    nft_token_id: str

    name: str
    image_url: str

    speed: int
```

```
strength: int
intelligence: int
resistance: int
rarity: int

family: str

level: int

created_at: datetime
expires_at: datetime

owner_id: UUID
```

6. Cálculo do Nível do Card

```
level = (
    speed +
    strength +
    intelligence +
    resistance +
    rarity
)
```

7. Sistema de Expiração

Regra Base

Validade média:

- 365 dias

Variação:

- -60 até +60 dias

Quanto maior a raridade:

- maior duração
-

Fórmula

```
base_days = 365
bonus = int((rarity - 50) * 1.2)
expiration_days = base_days + bonus
```

8. Geração Procedural de Cards

Processo Automatizado

Um worker executa periodicamente:

- criação de novos cards
- geração de NFTs
- upload de imagem
- persistência
- indexação

Estratégia de Geração

Distribuição de raridade

Tipo	Chance
Comum	50%
Raro	30%
Épico	15%
Lendário	5%

Garantia de Unicidade

Hash SHA256

```
hash = sha256(
    f"{name}-{speed}-{strength}-{intelligence}-{resistance}-{rarity}"
)
```

Se existir:

- regenera o card

9. Sistema NFT

Blockchain Recomendada

Polygon

Motivos:

- baixo custo
- compatível com Ethereum
- rápida
- excelente para games

Smart Contract

Padrão:

- ERC-721

Dados NFT

Metadados:

```
{
  "name": "Shadow Titan",
  "description": "Card raro do jogo",
  "image": "https://cdn.game.com/card.png",
  "attributes": [
    {"trait_type": "speed", "value": 82},
    {"trait_type": "strength", "value": 91}
  ]
}
```

10. Fluxo do Jogo

Preparação

1. Jogador escolhe 10 cards
 2. Cria sessão
 3. Matchmaking inicia
-

Matchmaking

Estratégia

Match por nível médio do deck.

```
player_deck_level = sum(card.level for card in deck)
```

Tolerância

Busca:

- ± 20 pontos

Fluxo

1. Busca jogador online
2. Caso encontre:
inicia partida PvP
3. Caso não encontre:
gera BOT equivalente

11. BOT Inteligente

Regras

BOT:

- possui deck semelhante
- utiliza IA simples
- escolhe melhor atributo probabilisticamente

Estratégia do BOT

```
best_attribute = max(  
    ["speed", "strength", "intelligence", "resistance", "rarity"],  
    key=lambda attr: getattr(card, attr)  
)
```

12. Estrutura da Partida

Entidades

```
class Match:
    id
    player1
    player2
    rounds
    winner
    status
```

Rodada

```
class Round:
    id
    selected_attribute
    player1_card
    player2_card
    winner
```

13. Sistema de Pontuação

Regras

- Vitória da rodada = +1 ponto
 - Maior número de vitórias ganha a partida
-

14. Sistema de Créditos

Recompensas

Resultado	Créditos
Vitória	1
Derrota	0

Possíveis Melhorias

Bônus

- streak de vitórias
 - eventos especiais
 - temporada
 - ranking alto
-

15. Loja de Cards

Funcionalidades

- comprar cards
 - renovar cards
 - ofertas especiais
 - packs temáticos
 - mercado entre jogadores
-

Estrutura

```
class ShopOffer:
    id
    card_id
    price
    expires_at
```

16. Sistema de Ranking

Modelo Recomendado

ELO/MMR

Fórmula Simplificada

```
new_rating = old_rating + k * (result - expected)
```

Divisões

Tier	Pontuação
Bronze	0-999
Silver	1000-1499
Gold	1500-1999
Platinum	2000-2499
Diamond	2500+

17. Sistema de Negociação de Cards via Blockchain

Objetivo

Permitir que jogadores:

- troquem cards
- negociem cards
- comprem/vendam cards
- construam reputação
- participem de uma economia descentralizada

Regras da Negociação

Cada negociação deverá:

- ser autenticada
- ser registrada em blockchain
- possuir rastreabilidade
- garantir autenticidade do NFT
- impedir duplicação ou fraude

Funcionalidades

Troca Direta

Jogador A oferece:

- 1 ou mais cards

Jogador B oferece:

- 1 ou mais cards
- créditos opcionais

A troca somente ocorre quando:

- ambos aprovarem
 - smart contract validar
-

Marketplace Secundário

Os jogadores poderão:

- listar cards para venda
 - definir preço
 - criar leilões
 - comprar cards de outros jogadores
-

Estrutura do Marketplace

```
class MarketplaceListing:
    id
    seller_id
    card_id
    price
    status
    created_at
    expires_at
```

Sistema de Oferta e Demanda

Funcionalidades

- ofertas públicas
 - wishlist
 - procura de cards específicos
 - recomendações automáticas
 - histórico de preços
 - tendência de mercado
-

Sistema de Reputação

Após cada negociação:

- comprador avalia vendedor
- vendedor avalia comprador

Métricas

Métrica	Descrição
Trust Score	reputação geral
Successful Trades	trocas concluídas
Fraud Reports	denúncias
Response Time	tempo de resposta

Benefícios da Blockchain

Segurança

- imutabilidade
- rastreabilidade
- autenticidade
- transparência

Smart Contracts

Responsáveis por:

- validar posse do NFT
- bloquear ativos durante negociação
- executar troca automática
- registrar transação

Fluxo da Negociação

1. Jogador cria oferta
2. Oferta é publicada
3. Outro jogador aceita
4. Smart contract valida ativos
5. Blockchain registra transação

6. Ownership dos NFTs é transferido
7. Sistema atualiza inventário

Eventos Blockchain

TradeCreated
TradeAccepted
TradeCancelled
NFTTransferred
MarketplaceListingCreated
MarketplaceSaleCompleted

Blockchain Recomendada

Polygon

Motivos:

- baixo custo de gas
- alta escalabilidade
- compatível com Ethereum
- excelente ecossistema gaming

Estratégia Recomendada

Custodial Wallet no MVP

Permite:

- onboarding simples
- menor fricção
- login social
- experiência semelhante a jogos tradicionais

Mercado Secundário

Possibilidades futuras:

- royalties para criadores
- marketplace global
- interoperabilidade
- staking de NFTs

- aluguel de cards
-

Anti-Fraude

Estratégias

- escrow via smart contract
 - cooldown de negociação
 - detecção de lavagem de ativos
 - monitoramento de preços suspeitos
 - análise comportamental
-

Integração com Economia do Jogo

Negociações podem utilizar:

- créditos internos
 - tokens blockchain
 - combinação híbrida
-

Eventos e Promoções

A blockchain permite:

- eventos exclusivos
 - drops raros
 - recompensas de traders ativos
 - campanhas limitadas
 - torneios especiais
-

18. Recursos Sociais

Funcionalidades

- amigos
 - convites
 - chat
 - guildas
 - compartilhar decks
 - rankings globais
 - replay de partidas
-

19. Deck Inicial

Regras

Primeiro acesso:

- recebe 9 cards aleatórios balanceados
 - recebe créditos para comprar o décimo
-

Limites

Deck inicial:

- sem lendários
 - raridade limitada
 - atributos limitados
-

20. APIs Principais

Auth

```
POST /auth/login
POST /auth/register
```

Cards

```
GET /cards
GET /cards/{id}
POST /cards/select-deck
```

Matchmaking

```
POST /matchmaking/find
GET /match/{id}
POST /match/{id}/play
```

Loja

GET /shop/offers
POST /shop/buy

Ranking

GET /ranking/global
GET /ranking/friends

21. Arquitetura de Microserviços

Serviços

Serviço	Responsabilidade
auth-service	autenticação
card-service	cards
nft-service	blockchain
matchmaking-service	partidas
gameplay-service	regras
ranking-service	ranking
economy-service	créditos
social-service	amigos/chat
notification-service	push

22. Comunicação Entre Serviços

Síncrona

- REST
- gRPC

Assíncrona

- RabbitMQ

23. Eventos de Domínio

Exemplos

```
CardCreated
MatchStarted
RoundFinished
PlayerWonMatch
CreditsEarned
CardExpired
PlayerRankUpdated
```

24. Banco de Dados

Tabelas Principais

players

```
CREATE TABLE players (
  id UUID PRIMARY KEY,
  nickname VARCHAR(50),
  rating INTEGER,
  credits INTEGER
);
```

cards

```
CREATE TABLE cards (
  id UUID PRIMARY KEY,
  owner_id UUID,
  name VARCHAR(255),
  speed INTEGER,
  strength INTEGER,
  intelligence INTEGER,
  resistance INTEGER,
  rarity INTEGER,
  level INTEGER,
  expires_at TIMESTAMP
);
```

25. Estratégia de Escalabilidade

Matchmaking em Redis

Fila:

```
queue:bronze  
queue:silver  
queue:gold
```

Cache

- ranking
- loja
- perfil
- cards populares

26. Segurança

Necessário

- JWT
- OAuth2
- rate limit
- anti-cheat
- validação server-side
- criptografia

27. Anti-Cheat

Estratégias

- lógica sempre no backend
 - validação de rodada
 - replay auditável
 - detecção de bots
 - monitoramento de padrões
-

28. Testes

Testes Unitários

Exemplo

```
def test_card_level_calculation():  
    card = Card(  
        speed=10,  
        strength=20,  
        intelligence=30,  
        resistance=40,  
        rarity=50  
    )  
  
    assert card.level == 150
```

Testes de Integração

Cenários

- criação de partida
- matchmaking
- compra de card
- atualização de ranking

Testes E2E

Fluxos

- login
 - selecionar deck
 - iniciar partida
 - jogar partida
 - comprar card
-

29. Estratégia Ágil

Scrum

Sprints

- 2 semanas
-

Roadmap MVP

Fase 1

- autenticação
- cards
- gameplay básico
- matchmaking

Fase 2

- ranking
- loja
- créditos

Fase 3

- NFTs
- blockchain
- marketplace

Fase 4

- social
 - guildas
 - eventos
-

30. DevOps

Pipeline

1. Pull Request
2. Testes
3. Build Docker
4. Scan segurança
5. Deploy staging

- 6. Testes E2E
- 7. Deploy produção

31. Observabilidade

Ferramentas

- Prometheus
- Grafana
- Loki
- OpenTelemetry
- Jaeger

32. Estratégia de Monetização

Possibilidades

- battle pass
- skins
- marketplace NFT
- packs premium
- cosméticos
- torneios pagos

33. Melhorias Futuras

Novos Modos

- torneios
- draft
- cooperativo
- boss battles
- guerras de guilda

IA Avançada

- recomendação de deck
 - matchmaking adaptativo
 - geração procedural inteligente
-

Eventos Dinâmicos

- temporadas
 - eventos limitados
 - cards sazonais
 - campanhas narrativas
-

34. Sugestões Estratégicas Importantes

1. Não utilizar blockchain no MVP

Primeiro validar:

- gameplay
- retenção
- matchmaking
- economia

Adicionar NFT posteriormente.

2. Priorizar gameplay competitivo

O sucesso do jogo depende:

- equilíbrio
 - matchmaking rápido
 - progressão divertida
-

3. Utilizar Feature Flags

Permite:

- liberar funcionalidades gradualmente
 - A/B testing
 - rollback rápido
-

4. Criar Telemetria Desde o Início

Medir:

- retenção
- abandono
- win rate

- balanceamento
 - economia
-

35. Estrutura Recomendada do Repositório Git

```
/game-platform
  /backend
  /frontend-web
  /frontend-mobile
  /infra
  /smart-contracts
  /docs
```

36. Estratégia de Balanceamento

Problema

Jogos de cards tendem a ficar desbalanceados.

Solução

Sistema de analytics:

- taxa de vitória por card
 - frequência de uso
 - sinergias
 - cards dominantes
-

Possíveis ações

- nerf
 - buff
 - rotação
 - expiração dinâmica
-

37. Arquitetura Recomendada para Tempo Real

Tecnologia

- WebSocket
-

Eventos em Tempo Real

- rodada iniciada
 - atributo escolhido
 - resultado
 - ranking atualizado
 - notificações
-

38. Sistema de Temporadas

Duração

- 30-90 dias
-

Benefícios

- reset parcial ranking
 - recompensas exclusivas
 - maior retenção
-

39. Marketplace Entre Jogadores

Funcionalidades Futuras

- venda de cards
 - leilão
 - troca
 - histórico de preço
-

40. Sistema de Energia (Opcional)

Pode limitar:

- número de partidas
 - farming abusivo
-

41. Conclusão

O projeto possui excelente potencial para:

- competição online
- coleção de NFTs
- economia digital
- expansão contínua
- eventos sazonais
- marketplace

A arquitetura proposta garante:

- escalabilidade
- modularidade
- facilidade de manutenção
- evolução futura
- suporte a milhões de jogadores

A combinação de:

- DDD
- microsserviços
- gameplay competitivo
- NFTs
- matchmaking inteligente
- economia balanceada

cria uma base sólida para um jogo moderno, escalável e altamente engajante.